

DeadlineDesk: A Campus Web System for Placement Rounds and Academic Assignment Deadlines

Project Proposal for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Aryan Gupta, CSED* Aksh Goyal, CSED[†] Naveen Bansal, CSED[‡]

August 16, 2026

Abstract

Placement application windows and assignment cutoffs at TIET are often communicated through WhatsApp forwards, shared drives, and scattered LMS notices. Students miss deadlines; teaching assistants apply late rules inconsistently. This proposal presents **DeadlineDesk**, a role-based web system that combines a placement track (companies, rounds, document checklists, reminders) with an academic dropbox (submission, late policy, similarity stub, TA grading). Success is measured by on-time completion and correct late-policy behaviour, with a working prototype targeted for Week 7.

1 Introduction / Problem Statement

Campus academic and placement calendars place concurrent pressure on students. Placement cells publish company-specific application windows and document requirements; departments publish assignment due times with late policies. In practice, these obligations travel through informal channels: forwarded messages, spreadsheets, and drive folders. Context is lost, reminders are uneven, and last-minute rushes are common.

The gap is the lack of a single, enforceable system for two related deadline workflows. On the placement side, students rarely have a structured checklist and reminder trail for each round. On the academic side, submissions may bypass the LMS; late acceptance is decided manually; grading status is hard to track. Nearby course projects do not address this: CaseRoom focuses on interview practice and scoring; College Mentorship focuses on long-term mentor matching; generic campus-resource tools do not model round checklists or late-policy submission states.

DeadlineDesk closes the gap with a deployable web product: shared authentication and roles; a Placement Track for companies, rounds, document checklists, and reminders; and an Academic Dropbox for assignments, file upload, automated late evaluation, a lightweight similarity check stub, and TA grading. The problem is locally relevant, actionable with mature web frameworks, and suitable for measurable evaluation within one semester, consistent with UCS503P expectations for time-to-value and continuous integration.

*Roll No: 1024030764; Email: agupta15_be24thapar.edu

[†]Roll No: 1024030766; Email: agoyal12_be24thapar.edu

[‡]Roll No: 1024030767; Email: nbansal13_be24thapar.edu

2 Objectives

Primary goal. Build a campus web system that makes placement-round readiness and assignment submission outcomes visible, enforceable, and testable by Week 7, then refine design artefacts and quality for the end-semester evaluation.

Specific objectives.

1. Implement role-based authentication for Student, TA/Faculty, and Placement Admin, verified by login tests for each role.
2. By Week 7, deliver one end-to-end placement path: create company/round → document checklist → reminder log entry ($T-24h$).
3. By Week 7, deliver one end-to-end academic path: publish assignment with late policy → submit → automatic late flag → similarity stub score → TA grade.
4. Maintain 100% agreement between automated late flags and the stated policy on a fixed black-box suite of at least five automated tests.
5. Keep the similarity check as an explicit stub (not a research NLP system); run CI (build and tests) and publish project documentation via the course MkDocs Pages pipeline.

3 Methodology

This is an engineering methodology: architecture, module development, integration, and validation.

3.1 System architecture and technology stack

We shall use a layered web architecture:

- **Presentation:** responsive web UI for student, TA, and admin portals.
- **Application:** API for authentication, CRUD operations, status transitions, and reminder scheduling hooks.
- **Domain:** shared entities (User, Role, Deadline, Document, Reminder, StatusHistory) and module entities (Company, Round, ChecklistItem; Assignment, Submission, Grade).
- **Persistence:** relational database (PostgreSQL; SQLite acceptable in the early prototype) with indexed deadline and status queries.

Planned stack (final choice by team delivery speed): Django or Next.js, GitHub Actions for CI, MkDocs with GitHub Pages from the UCS503P template, and Pytest or Jest for tests. The design relies on mature components rather than novel algorithms.

3.2 Module development

1. **Authentication and roles.** Session or JWT authentication with role claims and route guards.
2. **Placement Track.** An administrator publishes a company and round with open/close times. Round states: draft → published → open → closed → archived. Students complete required checklist items; readiness requires all required items to be complete.
3. **Academic Dropbox.** A TA sets the due time and late policy (reject / accept-with-penalty / grace-window). Late evaluation is a function of submission time, due time, and policy. Submission states: draft → submitted → under-review → graded (with an optional late flag).

4. **Reminders and audit.** Persist scheduled and sent reminder records; outbound email is a later increment.

3.3 Integration, CI/CD, and validation

Work proceeds in weekly increments: build, test, and demonstrate thin features. CI runs linting and unit tests on each push; documentation deploys through the template Pages pipeline; a staging URL supports the Week 7 demonstration.

Evaluation. Primary metric: on-time completion rate (OTCR)—the fraction of tracked deadlines where the required student action occurs before cutoff, taken from database timestamps. Secondary metrics: reminder effectiveness; full agreement of the late-policy test suite; median TA turnaround time. Pilot plan: 8–15 student users; staff roles may be simulated by the team if needed.

Risks and mitigations. Two-module scope is controlled by a shared core and thin Week 7 paths. Reminder delivery on free hosting starts as a database log. The similarity component is labelled as a stub in the SRS. Official college email addresses are recorded on the team declaration sheet.

Scalability. Pagination, indexes, and clear module boundaries support growth in users and deadlines. Deployment targets common web hosting with a managed database and CI/CD.

4 Timeline

Week	Task	Deliverable
W3–W4	Problem definition, proposal, presentation, repository setup	Proposal PDF, slides, Pages site
W5–W6	SRS draft, use-case and class sketches, backlog, auth scaffold	SRS v0.1, CI green
W7	Both thin end-to-end paths and automated tests	Prototype (MST)
W8	Improvement plan from prototype feedback	Improvement note
W9–W10	Mid-semester buffer; defect fixes	Stable staging build
W11–W12	Sequence and state diagrams; additional tests; UI polish	Design pack v1
W13	Buffer (holiday sandwich)	Catch-up
W14–W15	Second prototype polish	Prototype 2
W16	Improvements over second prototype	Updated demonstration
W17	Final prototype, seminar, final report	EST delivery

Buffer: Weeks 13 and the mid-semester period absorb slippage. If needed, email reminders and dashboards are deferred before core status machines.

5 Resources / Budget

Human resources.

- **Aryan Gupta (1024030764):** architecture, CI/CD, placement module.
- **Aksh Goyal (1024030766):** academic dropbox module and late-policy tests.
- **Naveen Bansal (1024030767):** user interface flows, documentation site, demonstration script.

Role emphasis may adjust after Week 7 according to measured velocity.

Material resources. Personal laptops; GitHub (repository, Actions, Pages); free-tier hosting and database for staging; L^AT_EX for reports. No specialised laboratory hardware is required.

Budget. No external funding is required. Software is free and open-source or available through student GitHub access. Optional printing of a one-page handout is incidental.

6 Summary

DeadlineDesk is a focused software engineering project: two deadline workflows, clear roles, explicit status machines, continuous integration, and measurable on-time and policy-correctness metrics. It matches UCS503P selection heuristics—local relevance, available engines, deployability, and evaluation—and is attainable for a three-member team within the semester.